

Data Flow Diagram

This diagram shows how data flows through the BDI Kernel system from source code to execution.

Complete Data Flow

flowchart TD

```

A[Source Code] --> B[Lexer]
B --> C[Token Stream]
C --> D[Parser]
D --> E[Abstract Syntax Tree]
E --> F[Semantic Analyzer]
F --> G[Typed AST]
G --> H[IR Generator]
H --> I[Graph IR]
I --> J[Graph Optimizer]
J --> K[Optimized IR]
K --> L[Code Generator]
L --> M[Bytecode]

M --> N{Execution Path}

N -->|Interpretation| O[Interpreter]
N -->|JIT Compilation| P[JIT Compiler]

O --> Q[Stack Operations]
Q --> R[Memory Access]
R --> S[Result]

P --> T[LLVM IR]
T --> U[LLVM Optimizer]
U --> V[Native Code]
V --> W[Direct Execution]
W --> S

R --> X[Garbage Collector]
X --> Y[Heap Management]
Y --> R

```

```

style A fill:#e1f5ff
style M fill:#fff4e1
style S fill:#e1ffe1
style V fill:#ffe1e1

```

Detailed Data Transformations

1. Source to Tokens

```

flowchart LR
    A["Source: 'x = 5 + 3'"] --> B[Lexer]
    B --> C["Tokens:  
IDENTIFIER(x)  
EQUALS  
NUMBER(5)  
PLUS  
NUMBER(3)"]
  
```

2. Tokens to AST

```

flowchart TD
    A[Token Stream] --> B[Parser]
    B --> C["AST:  
Assignment  
  Identifier: x  
  BinaryOp: +  
    Literal: 5  
    Literal: 3"]
  
```

3. AST to Typed AST

```

flowchart TD
    A[AST] --> B[Type Checker]
    B --> C["Typed AST:  
Assignment (void)  
  Identifier: x (int)  
  BinaryOp: + (int)  
    Literal: 5 (int)  
    Literal: 3 (int)"]
  
```

4. AST to Graph IR

```

flowchart LR
    A[Typed AST] --> B[IR Generator]
    B --> C["Graph IR:  
[Const 5] --> [Add] --> [Store x]  
[Const 3] --> [Store x]"]
  
```

5. Graph Optimization

```

flowchart LR
    A["Before:  
[Const 5] --> [Add]  
[Const 3] --> [Store x]"] --> B[Optimizer]
    B --> C["After:  
[Const 8]"]
  
```

6. IR to Bytecode

```

flowchart LR
    A["Graph IR:  
[Const 8]  [Store x]"] --> B["Code Generator"]
    B --> C["Bytecode:  
PUSH 8  
STORE x"]
  
```

7. Bytecode to Native Code

```

flowchart LR
    A["Bytecode:  
PUSH 8  
STORE x"] --> B["JIT Compiler"]
    B --> C["Native Code:  
mov rax, 8  
mov [x], rax"]
  
```

Data Structures

Token Structure

```

typedef struct {
    TokenType type;
    const char* lexeme;
    int line;
    int column;
} Token;
  
```

AST Node Structure

```

typedef struct ASTNode {
    NodeType type;
    TypeInfo* type_info;
    SourceLocation location;
    union {
        double literal_value;
        char* identifier;
        struct {
            struct ASTNode* left;
            struct ASTNode* right;
        } binary_op;
    } data;
} ASTNode;
  
```

Graph Node Structure

```
typedef struct GraphNode {
    uint32_t id;
    NodeType type;
    struct GraphNode** inputs;
    struct GraphNode** outputs;
    TypeInfo* type;
} GraphNode;
```

Bytecode Instruction

```
typedef struct {
    OpCode opcode;
    uint8_t operands[8];
} Instruction;
```

Memory Flow

```
flowchart TD
    A[Allocation Request] --> B[Object Size]
    B -->|Small| C[Young Generation]
    B -->|Large| D[Old Generation]
    C --> E[Eden Space]
    E --> F[Space Available?]
    F -->|Yes| G[Bump Pointer Alloc]
    F -->|No| H[Minor GC]
    H --> I[Copy Live Objects]
    I --> J[Survivor Space]
    J --> K[Age >= Threshold?]
    K -->|Yes| L[Promote to Old Gen]
    K -->|No| E
    L --> D
    D --> M[Free List Alloc]
    G --> N[Allocated Object]
    M --> N
```

Control Flow

```

flowchart TD
    A[Program Entry] --> B[Initialize VM]
    B --> C[Load Bytecode]
    C --> D[Start Execution]

    D --> E{Instruction Type}

    E -->|Arithmetic| F[Compute Result]
    E -->|Branch| G[Evaluate Condition]
    E -->|Call| H[Push Call Frame]
    E -->|Return| I[Pop Call Frame]

    F --> J[Update Stack]
    G --> K[Update IP]
    H --> L[Jump to Function]
    I --> M[Restore Context]

    J --> N{More Instructions?}
    K --> N
    L --> N
    M --> N

    N -->|Yes| E
    N -->|No| O[Return Result]
  
```

Optimization Data Flow

```

flowchart TD
    A[Input Graph] --> B[Dead Code Elimination]
    B --> C[Constant Folding]
    C --> D[CSE]
    D --> E[Loop Optimization]
    E --> F[Inlining]
    F --> G[Strength Reduction]
    G --> H[Optimized Graph]

    style A fill:#ffcccc
    style H fill:#ccffcc
  
```

Profiling Data Flow

```
flowchart LR
    A[Execution] --> B[Profiler]
    B --> C[Execution Count]
    B --> D[Execution Time]
    B --> E[Memory Usage]

    C --> F[Hot Path Detector]
    D --> F

    F --> G{Is Hot?}
    G -->|Yes| H[Trigger JIT]
    G -->|No| I[Continue Profiling]

    H --> J[Compile to Native]
    J --> K[Execute Native Code]

    I --> A
    K --> A
```

[Back to Architecture Overview](#) (../README.md)